# Introduction to Machine Learning

Mihir Rao

## Contents

**1 Introduction and ML Problem Framing** **2**
- 1.1 Learning from data . . . 2
- 1.2 Supervised learning and decision regions . . . 2
- 1.3 Generalization and evaluation . . . 4
- 1.4 Other learning settings . . . 5
- 1.5 The machine-learning lifecycle . . . 5

**2 Data Tools** **6**
- 2.1 Tabular data and inspection . . . 6
- 2.2 Selecting and filtering . . . 7
- 2.3 Changing and cleaning data . . . 7
- 2.4 Aggregation and grouping . . . 8
- 2.5 Joining tables . . . 8
- 2.6 Visualization . . . 9

# Lecture 1: Introduction and ML Problem Framing

## 1.1 Learning from data

Machine learning builds systems that improve through data or experience. It is useful when desired behavior is difficult to specify as rules but examples or an evaluation of the result are available, as in spam filtering. Training uses data to fit a model; inference applies the fitted model to a new input. Artificial intelligence is a broader goal that also includes approaches such as symbolic reasoning and planning.

A model $h_w \colon X \to Y$ has parameters $w$ learned from data or interaction. Input representation, target, model family, and evaluation criterion all affect what can be learned.

## 1.2 Supervised learning and decision regions

In supervised learning, training examples have inputs and targets, $\{(x_i, y_i)\}_{i=1}^{n}$. Each row is a sample; the components of $x_i$ are features. Classification predicts a discrete class, while regression predicts a continuous target. A binary classifier partitions feature space into regions assigned to classes 0 and 1.

Consider the taste-test data below, where $A$ is acidity, $S$ is sweetness, and $y = 1$ means great taste. A numerical taste score would instead make this a regression task.

| Beverage | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $A$ | 0.8 | 0.3 | 0.2 | 0.3 | 0.9 |
| $S$ | 0.8 | 0.25 | 0.8 | 0.7 | 0.7 |
| $y$ | 1 | 0 | 0 | 0 | 1 |

A classifier assigns a label to every point $(A, S)$, not just the five observed points. Its decision boundary separates regions receiving different predictions. In the plots, filled points have label 1 and open points have label 0; shading shows the region predicted as 1.

A DECISION STUMP makes one threshold test. The rule $f(A, S) = \mathbf{1}\{S \geq 0.75\}$ predicts $(1, 0, 1, 0, 0)$, so its training accuracy is $3/5$. The horizontal boundary shows why: the stump ignores acidity, so the high-sweetness negative example is misclassified.

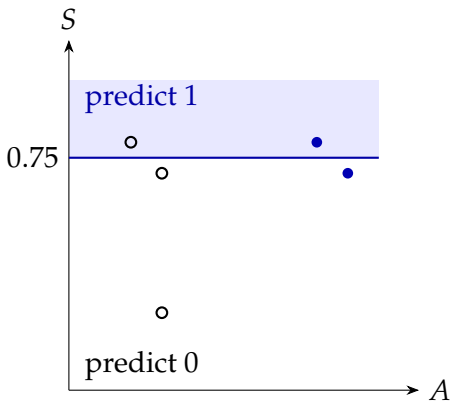



**Figure 1:** The stump splits on sweetness at $S = 0.75$.

A decision tree applies tests in sequence. Each path from the root to a leaf specifies a region, and the leaf supplies its prediction. For fixed splits, assigning the majority training label to each leaf maximizes training accuracy. The tree below gets all five labels right, although its second split does not change the prediction: both of its leaves predict 1.

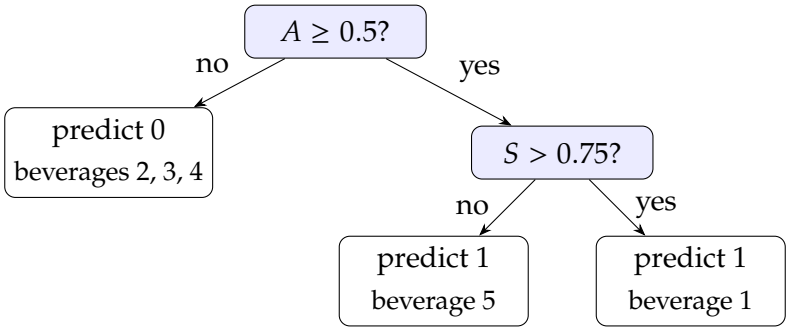



**Figure 2:** A decision tree for the taste-test data.

A LINEAR CLASSIFIER uses a straight decision boundary. For $f(A, S) = \mathbf{1}\{A + S - 1 \geq 0\}$, the boundary is $A + S = 1$ and the predictions are $(1, 0, 1, 1, 1)$, giving 3/5 accuracy. Raising the threshold to 1.3 shifts the parallel boundary and gives $(1, 0, 0, 0, 1)$, fitting all five. The rule predicts 1 on the boundary as well as above it. Unlike a tree's axis-aligned tests, this rule trades off acidity and sweetness along a slanted boundary.

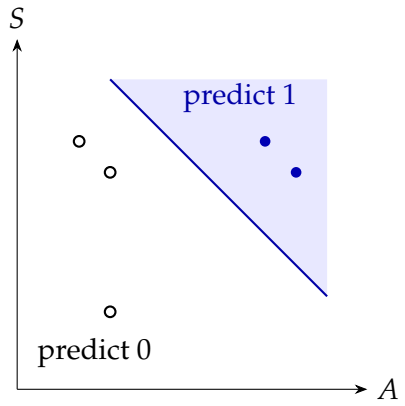



**Figure 3:** A linear boundary at $A + S = 1.3$.

### 1.3 Generalization and evaluation

The aim is to predict for new inputs, not merely reproduce training labels. For a classifier, training accuracy is the fraction of training predictions that match their labels; 0–1 loss is the fraction that do not. A held-out test set assesses performance on examples not used to fit or select the model. The evaluation is informative about future use only to the extent that those examples represent the intended deployment setting.

A lookup rule can memorize every training example and output zero elsewhere. If identical feature vectors have consistent labels, it has perfect training accuracy, yet its predictions on unseen points need not be useful. In particular, it fails on unseen positive examples; its actual test accuracy depends on the test distribution. Good training fit alone therefore does not establish GENERALIZATION. Simpler models can express useful assumptions about unseen data, but simplicity alone does not guarantee success.

## 1.4 Other learning settings

Unsupervised learning uses inputs without explicit target labels. Clustering groups similar observations; density estimation models how data are distributed; dimensionality reduction seeks a lower-dimensional representation that retains useful structure. Other examples include anomaly detection and data generation. A reduced representation can support visualization or subsequent learning tasks.

Reinforcement learning learns behavior from rewards, often over a sequence of actions. Unlike a labeled input-output example, the reward may evaluate a sequence or outcome, so choosing actions requires relating feedback to earlier decisions.

## 1.5 The machine-learning lifecycle

1. **Frame the learning problem.** Specify the target and objective, choose a measure of success, identify available data and features, and split training and test data.
2. **Design the model.** Choose a model family or architecture and its hypothesis space. Its inductive biases are assumptions that favor some solutions over others.
3. **Optimize.** Define a loss and choose an optimization method to fit the model. Manage regularization and overfitting.
4. **Predict and evaluate.** Apply the model and measure performance with metrics suited to the objective. Check whether the model works for the intended reasons and population, and use the results to revisit earlier choices.

**Source.** Jennifer Listgarten and Alex Dimakis, Introduction to Machine Learning, Spring 2026, Lecture 1.

# Lecture 2: Data Tools

## 2.1 Tabular data and inspection

pandas organizes and transforms tabular data. A Series is a one-dimensional array with an index of labels; a DataFrame is a two-dimensional table with labeled rows and columns. A column selected by name is a Series. Labels identify data, while integer positions describe its current order. The two need not be the same.

The examples below use the landmark data. Missing construction years are represented by `None` on input and treated as missing by pandas.

```
import pandas as pd

df = pd.DataFrame({
    "Landmark": ["Sather Gate", "Campanile", "Doe Library",
                 "Memorial Glade", "Sproul Plaza"],
    "Type": ["Gate", "Tower", "Library", "Open Space", "Plaza"],
    "Height": [30, 307, 80, 0, 0],
    "Year Built": [1910, 1914, 1911, None, 1962],
})
# Alternatively: pd.read_csv("data/uc_berkeley_events.csv", index_col="Year")
```

Before transforming data, inspect its shape, types, missing values, and representative rows. `df.shape` gives (rows, columns); `df.size` counts cells. For this table they are `(5, 4)` and `20`. `info()` reports column types and non-null counts; `describe()` summarizes numerical columns by default.

```
df.head()                          # first 5 rows; tail(3) gives last 3
df.sample(n=2, random_state=42)     # reproducible random rows
df["Type"].value_counts()          # frequencies of non-missing values
df["Type"].unique()                # distinct values in appearance order
```

Sampling also accepts `frac` for a fraction of rows and `replace=True` to sample with replacement.

## 2.2 Selecting and filtering

Use `iloc` for zero-based integer positions and `loc` for labels. Both accept a scalar, a list, or a slice for each axis. Position slices exclude the stop, as in Python; label slices include both endpoints when the labels are present. A single selected column is a Series, while a list of columns retains a DataFrame.

```
df.iloc[2:4, 1:3]                # positions 2, 3 and columns 1, 2
df.loc[2:3, "Type":"Height"]    # labels 2, 3; both column endpoints
df["Height"]                     # Series
df[["Landmark", "Height"]]       # DataFrame
```

Here the default row labels happen to match positions. After sorting or changing the index, `loc[2]` still selects label 2, whereas `iloc[2]` selects the third row. Bracket selection is context-dependent: a column name selects a column, while `df[1:3]` slices rows.

A Boolean condition produces a mask aligned with the rows; selecting with it keeps the true entries. Combine conditions with & (and), | (or), ~ (not), or ^ (exclusive or), enclosing each comparison in parentheses.

```
tall = df["Height"] > 50
df.loc[tall, ["Landmark", "Height"]]
df[(df["Height"] > 50) & (df["Type"] == "Library")]
```

## 2.3 Changing and cleaning data

Assign a new column from an appropriately sized array or an expression on existing columns. Most operations return a new object rather than changing the original, so assign the result when the change should persist.

```
df["Height Increase"] = df["Height"] * 0.1
df = df.drop(columns=["Height Increase"])
df = df.sort_values(by=["Height", "Type"], ascending=[True, False])
df["Year Built"].isna()           # detect missing entries
df.dropna(subset=["Year Built"])  # remove rows missing this field
df["Year Built"].fillna(1900)     # illustrative replacement, not a fact
```

Dropping rows can discard useful data; imputation replaces missing values and introduces assumptions. Choose the treatment for the task, and do not mistake a filled value for an observation.

## 2.4 Aggregation and grouping

An aggregation summarizes many values with one result. For example, `df["Height"].mean()` is 83.4, and `df["Height"].idxmax()` returns label 1, the index of the maximum, not the maximum height itself. Other operations include `sum`, `median`, `std`, `count`, and `nunique`.

GROUP BY splits rows by key, applies an aggregation within each group, and combines the results. The grouping keys identify the resulting rows; selecting a value column specifies what is summarized.

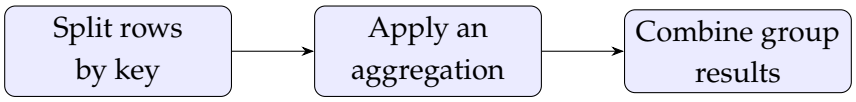



**Figure 4:** Grouping follows a split, apply, combine pattern.

```
df.groupby("Type")["Height"].agg("mean")
# With a Campus column, group by two keys:
# augmented_df.groupby(["Type", "Campus"])["Height"].max()
```

A PIVOT TABLE arranges group summaries in a grid. For example, `pd.pivot_table(augmented_df, index="Type", columns="Campus", values="Height", aggfunc="max")` places types in rows and campuses in columns, with maximum height in each cell.

## 2.5 Joining tables

A join combines tables using matching keys. An INNER JOIN retains matches; a LEFT JOIN retains all left rows, and a right join retains all right rows. An OUTER JOIN retains rows from both sides, filling unmatched fields with missing values. If a key occurs more than once on each side, matching rows combine in all pairs, so the result can have more rows than either input.

```
# event_data has columns "Year" and "Event".
merged = df.merge(event_data, how="left",
                  left_on="Year Built", right_on="Year")
# Match the same keys using the right table's index:
joined = df.join(event_data.set_index("Year"),
                 on="Year Built", how="left")
```

Choose the join according to which unmatched rows matter; check keys and row counts. pandas matches missing keys to each other, unlike ordinary SQL joins.

## 2.6 Visualization

Plots help reveal structure and communicate it. Scatter plots show relations between two variables, line plots show change along an ordered axis, histograms show a numerical distribution, bar plots compare categories, heatmaps show a grid of values, and box plots summarize distributions. Matplotlib commonly produces static figures; Plotly supports interactive inspection. In either case, map columns to visual properties such as position, color, and size.

```
import plotly.express as px

# mpg is a DataFrame with weight, mpg, origin, and cylinders columns.
fig = px.scatter(mpg, x="weight", y="mpg", color="origin",
                 size="cylinders", hover_data=mpg.columns,
                 title="MPG vs. Weight by Origin")
fig.show()
```

Plotly Express offers concise plotting functions; graph objects allow more explicit construction. pandas can also use Plotly as its plotting backend: `pd.set_option("plotting.backend", "plotly")`, followed by `mpg.plot(kind="scatter", x="weight", y="mpg")`.

**Source.** Jennifer Listgarten and Alex Dimakis, Introduction to Machine Learning, Spring 2026, Lecture 2.